

Manual del flujo de reducción y análisis multi-método

ROXs 12 B con VLT/MUSE — secuencia operativa y catálogo de códigos

Julio 2026 · repositorio MUSE-accretion-pipeline

Este documento describe el flujo de trabajo completo del pipeline, asumiendo el plan maestro (docs/plan_roxs12_reduccion_multimetodo.md) ya implementado en su totalidad. Tiene dos partes. La **Parte I** recorre la secuencia de ejecución etapa por etapa, con las funciones y comandos canónicos, sus entradas, sus productos y los puntos de control donde el flujo se detiene a esperar revisión humana. La **Parte II** es un catálogo de los módulos de código: qué hace cada uno, qué recibe y qué entrega, sin entrar en detalles de implementación.

Convenciones globales. Todos los productos de una ejecución viven en runs/<RUN_ID>/{config, stages, tables, plots, logs, report}. El run activo se resuelve con la variable de entorno MUSE_RUN_ID o, en su defecto, con active_run.txt. Cada etapa escribe un archivo de control de calidad stageXX_qc.json con parámetros efectivos, hashes de sus entradas y métricas de éxito; ninguna etapa posterior consume un producto sin verificar ese QC. Los espectros extraídos comparten un formato único (*SpectrumProduct*) que hace comparables todos los métodos.

Puntos de entrada. El campo entry_point de la configuración admite tres valores: raw (reducción completa desde datos crudos ESO, bloque A entero), eso_cube (cubo ya reducido, p. ej. el ADP del archivo; el bloque A se reduce a las etapas condicionales y al QC) y cropped_cube (compatibilidad con los runs históricos). La cadena B–F es idéntica en los tres casos.

Parte I · Secuencia de trabajo

Las etapas se ejecutan en el orden listado. Las marcadas (*condicional*) deciden por métricas si actúan o se saltan; ambos desenlaces son válidos y quedan registrados. Las marcadas con ● incluyen un punto de control humano: el flujo se detiene y presenta evidencia antes de continuar.

Bloque A — Reducción y calidad del cubo

A1 · Reducción desde datos crudos (solo entry_point=raw)

```
bash scripts/reduce_raw.sh
```

Entradas: Datos crudos ESO (ciencia + calibraciones asociadas por calselector), esorex con el pipeline MUSE instalado.

Qué hace: Recorre la cascada estándar (bias, flat, wavecal, LSF, scibasic, standard, scipost) con parámetros por defecto, construyendo los SOF programáticamente y validándolos antes de cada recipe. Guarda pixtables intermedios para poder rehacer el cielo sin repetir todo. Al final ejecuta seis verificaciones, incluida la comparación contra el cubo ADP de referencia (imagen blanca y espectro de la primaria).

Productos: `DATA_CUBE_FINAL.fits` (DATA+STAT), `pixtables`, `stage00r_qc.json`, figuras.

● **Punto de control:** Dos: aprobación de la lista de descarga del archivo ESO y revisión de los SOF antes de lanzar scipost.

A2 · Limpieza de residuales de cielo con ZAP (condicional)

```
bash scripts/sky_zap.sh
```

Entradas: Cubo de A1 o cubo ADP (procedencia registrada en QC); posiciones aproximadas de las fuentes del campo.

Qué hace: Mide la métrica $R = \text{rms}(\text{skylines})/\text{rms}(\text{continuo})$ en aperturas vacías. Si $R \leq 1.5$ la etapa se salta con evidencia; si $R > 2$ construye una máscara de fuentes (con inclusión forzosa de primaria, compañero y tercera fuente) y aplica ZAP con parámetros por defecto. Verifica que los residuales bajaron sin alterar el espectro de las fuentes fuera de las ventanas de cielo, y que la región de $H\alpha$ quedó intacta.

Productos: `cube_zap.fits` (si se aplicó), máscara de fuentes, `stage00s_qc.json` con la decisión y sus números.

● **Punto de control:** Revisión de la figura de máscara + métrica R antes de ejecutar ZAP; también en la zona gris de decisión ($1.5 < R \leq 2$).

A3 · Corrección telúrica con molecfit (condicional)

```
bash scripts/telluric.sh
```

Entradas: Cubo de A2 (o A1/ADP), espectro de la primaria como referencia de ajuste.

Qué hace: Mide la profundidad telúrica en las bandas de O2 y H2O sobre el espectro de la primaria. Si es $< 3\%$ o la ciencia no usa el continuo rojo, se salta. Si actúa, ajusta molecfit sobre regiones libres de líneas estelares y aplica la transmisión 1D a todo el cubo, protegiendo las ventanas de $H\alpha$ y del láser AO (transmisión forzada a 1). STAT se escala por transmisión al cuadrado.

Productos: `cube_telcorr.fits` y la transmisión (`TELLURIC_TRANS.fits`), `stage00t_qc.json`.

● **Punto de control:** Revisión del ajuste por región (dato, modelo, residuo, χ^2) antes de aplicar al cubo.

A4 · QC del cubo — puerta de entrada común (todos los entry points)

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.qc.cube_qc import run_stage00q; run_stage00q()"
```

Entradas: Cubo de trabajo (re-reducido o ADP; si hubo ZAP, también el cubo pre-ZAP para las skylines), catálogo de skylines, fotometría Gaia DR3 de la primaria.

Qué hace: Cinco mediciones con semáforo verde/amarillo/rojo: M1 solución de longitud de onda por skylines (teniendo en cuenta el marco baricéntrico aplicado por scipost); M2 LSF empírica; M3 escala de flujo por fotometría sintética vs. Gaia; M4 estadística de cielo; M5 validación de la extensión STAT en dos escalas espaciales. No corrige nada: entrega factores que D2 y E1 consumirán.

Productos: stage00q_qc.json con $\Delta\lambda$, LSF(λ), factor de flujo, factores STAT y semáforos; figuras por medición.

● **Punto de control:** Cualquier medición en rojo detiene el flujo (p. ej. STAT rojo cambia la estrategia de errores de toda la cadena al modo empírico).

Bloque B — Preparación y localización

B1 · Carga, alineación y recorte

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage01; run_stage01()"
```

Entradas: Cubo(s) validado(s) por A4; configuración de recorte (crop_npix, método de centrado).

Qué hace: Alinea exposiciones (si hay varias), recorta el campo de trabajo alrededor de la primaria y apila. Propaga la extensión STAT por las mismas transformaciones y registra los desplazamientos aplicados y el factor de covarianza espacial que introduce la interpolación.

Productos: stage01_cube_stack.fits (DATA+STAT), stage01_qc.json con shifts y factores.

B2 · Corrección de stripes

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage02; run_stage02()"
```

Entradas: Cubo apilado de B1.

Qué hace: Corrige el patrón de offsets espectrales entre slices por correlación cruzada (la maquinaria validada de musepipe/stripes.py), aplica los mismos desplazamientos a STAT y escribe una métrica escalar de amplitud de stripes por canal, con la lista de canales sucios que las etapas de H α consultarán.

Productos: stage02_xcorr_cube_stack.fits, tabla de métrica por canal, stage02_qc.json.

B3 · Localización del target y astrometría

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage01c; run_stage01c()"
```

Entradas: Cubo de B2; separación y ángulo de posición esperados (literatura) en la configuración.

Qué hace: Mide las posiciones de la primaria, del compañero y de la tercera fuente. Al compañero lo busca por matched filter solo dentro de una región de tolerancia alrededor de la posición predicha (nunca búsqueda ciega). Convierte a separación/PA, compara con la predicción orbital y mide la deriva cromática del centroide, que decide si los extractores usan centroide dependiente de λ . Las coordenadas medidas pasan a ser las canónicas: ninguna etapa posterior usa números escritos a mano.

Productos: stage01c_qc.json con posiciones (con incertidumbre), astrometría vs. literatura y veredicto cromático; figuras de verificación.

● **Punto de control:** Si aparecen dos candidatos en la tolerancia, o el compañero no se detecta en continuo, el flujo se detiene (esto último cambiaría la estrategia científica).

Bloque C — PSF cromática y extracciones

C1 · Modelo de PSF cromática

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_e01; run_stage_e01()"
```

Entradas: Cubo de B2, posiciones de B3, LSF de A4.

Qué hace: Ajusta un perfil Moffat elíptico (y maoppy como contraste) a la primaria en bins de $\sim 100 \text{ \AA}$, con el compañero y la tercera fuente enmascarados, suaviza los parámetros en λ y empaqueta todo como una función evaluable `psf_model( $\lambda$ , dy, dx)` normalizada. La métrica que gobierna la etapa es el residuo del modelo en el anillo del compañero: es el error sistemático que heredan las extracciones. Si supera el umbral, añade una componente híbrida azimutal que no puede absorber fuentes puntuales.

Productos: `psf_model.json`, tabla de parámetros por bin, `stage_e01_qc.json` con la métrica del anillo.

● **Punto de control:** Si la métrica sigue fuera de umbral tras el híbrido, el flujo se detiene con el mapa de residuo y opciones.

C2 · Extracción por apertura + fondo local (método de control)

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_x01; run_stage_x01()"
```

Entradas: Cadena de sustracción local validada (04b) sobre el cubo de B2; coordenadas de B3; curva de crecimiento de C1.

Qué hace: Envuelve la extracción histórica (superficie local + suma en apertura 3×3 y 5×5) en el formato estándar, añadiendo errores propagados desde STAT (con los factores de A4/B1), el error empírico de controles como columna paralela y la corrección de apertura por canal. Reproduce numéricamente el baseline validado del repositorio.

Productos: `spec_aperture_object.fits` (formato estándar), `spec_aperture_qc.json`.

C3 · Extracción óptima (Horne)

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_x02; run_stage_x02()"
```

Entradas: Cubo con fondo restado (dos variantes: superficie local y modelo de primaria de C1), psf_model, STAT, coordenadas de B3.

Qué hace: Pondera los píxeles de la ventana de extracción por el perfil PSF esperado y la varianza, canal a canal, con rechazo espacial de outliers que no puede recortar líneas reales. Devuelve flujo y varianza analítica; la ganancia típica de S/N sobre la apertura es del 10–30% sin sesgo de continuo.

Productos: spec_optimal_object.fits y spec_optimal_psfsub_object.fits, spec_optimal_qc.json.

C4 · Ajuste PSF simultáneo estrella + compañero (método primario)

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_x03; run_stage_x03()"
```

Entradas: Cubo de B2 sin sustracción previa, psf_model, posiciones de B3, STAT.

Qué hace: Resuelve por mínimos cuadrados lineales, canal a canal, el modelo dato = a·PSF(estrella) + b·PSF(compañero) + plano de fondo. Los errores salen de la matriz de covarianza del ajuste y se validan repitiendo el ajuste en las posiciones de control. Diagnostica el condicionamiento y el crosstalk (líneas de la primaria filtrándose al compañero). El espectro de la primaria ajustada sirve de control de escala absoluta.

Productos: spec_psffit_object.fits, spec_psffit_star.fits, cubo residual del ajuste, spec_psffit_qc.json.

Bloque D — Comparación y calibración

D1 · Comparación entre métodos (árbitro)

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_x10; run_stage_x10()"
```

Entradas: Todos los espectros en formato estándar (C2, C3, C4) y sus controles.

Qué hace: Compara los métodos por canal y en nueve bandas congeladas (seis de continuo, tres de línea), usando como error de cada diferencia la dispersión empírica medida sobre los controles (los métodos comparten fotones, así que los errores propagados sobrestimarían la incertidumbre de la diferencia). Emite un veredicto automático: consistente, divergencia de continuo (→ refinar PSF, no saltar a PCA), divergencia de líneas (→ tratar en el bloque E) o ininterpretable (controles no limpios).

Productos: tables/method_comparison.csv, stage_x10_qc.json con el veredicto, figura panel.

● **Punto de control:** Siempre: el veredicto y la recomendación se presentan y el usuario elige el método canónico que continúa la cadena.

D2 · Calibración del espectro final

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_x11; run_stage_x11()"
```

Entradas: Espectro canónico elegido en D1 (y los demás, para consistencia); factores medidos por A4; sistemáticos declarados por A2/A3/C1.

Qué hace: Aplica las correcciones medidas aguas arriba — offset de longitud de onda, factor de flujo — tras una auditoría explícita contra el doble conteo (marco baricéntrico ya aplicado, corrección de apertura ya incluida, factores STAT ya dentro del error). Estima el continuo por dos vías independientes (mediana móvil y polinomio con máscara de líneas), combina el error estadístico con la regla conservadora max(propagado, empírico) suavizada, y tabula el presupuesto completo de sistemáticos, cada término con su fuente.

Productos: `spec_final_object.fits` con columnas de continuo y error total, `stage_x11_qc.json` con el presupuesto.

Bloque E — Análisis de H α

E1 · Detección y significancia

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_h01; run_stage_h01()"
```

Entradas: Espectros calibrados de todos los métodos con sus controles; LSF de A4; velocidad radial sistémica de la configuración.

Qué hace: Aplica un matched filter en velocidad alrededor de H α con tres anchos de plantilla, calcula la significancia empírica contra la distribución de los controles y corrige por búsqueda (look-elsewhere) barriendo todos los canales buenos y las tres plantillas. El criterio de detección está congelado de antemano: FAP global < 1% en al menos dos métodos con fondo independiente, y centroide compatible con la velocidad sistémica. El orden es obligatorio: primero los controles, después el objeto.

Productos: `tables/halpha_detection_by_method.csv`, `stage_h01_qc.json` con el veredicto, figuras por método.

● **Punto de control:** Si el resultado es “candidato” (señal en un solo método), se decide en conjunto el orden de las pruebas de E2.

E2 · Batería de pruebas de artefactos

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_h02; run_stage_h02()"
```

Entradas: Resultado de E1, cubos residuales, métrica de stripes de B2, catálogo de skylines.

Qué hace: Ejecuta siempre la batería completa de cinco pruebas: coincidencia con artefactos instrumentales, coherencia espacial con la PSF, estabilidad temporal en sub-stacks (si hay varias exposiciones), estabilidad frente a parámetros y líneas placebo por la misma cadena estadística. Una detección solo sobrevive si pasa todas las aplicables; una no-detección es robusta si ningún placebo supera el umbral.

Productos: `stage_h02_qc.json` con veredicto por prueba y global; figuras (stamp vs. PSF, tornado de parámetros, placebos).

E4 · Inyección y recuperación multi-método

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_h04; run_stage_h04()"
```

Entradas: Cubo de B2 (clonado en copias aisladas), psf_model, extractores C2–C4, estimador de E1.

Qué hace: Inyecta compañeros sintéticos con la PSF medida (línea H α de amplitud, ancho y continuo variables) en la posición real y en las de control, propaga cada clon por la cadena completa y mide throughput, sesgos y completitud por método. Un subconjunto se repite con la PSF perturbada $\pm 10\%$ para acotar la sensibilidad al modelo. Incluye la regresión contra la validación histórica de inyección del repositorio.

Productos: tables/injection_throughput_by_method.csv, curvas de throughput y completitud, stage_h04_qc.json.

● **Punto de control:** El presupuesto de cómputo se aprueba antes de lanzar la grilla completa.

E3 · Límites superiores (si no-detección)

```
MUSE_RUN_ID=<run> python -c \  
"from musepipe.stages import run_stage_h03; run_stage_h03()"
```

Entradas: Distribuciones de controles de E1, throughput de E4 (obligatorio), calibración de D2, parámetros físicos con cita (distancia, extinción, relación L_{acc}–L_{H α}).

Qué hace: Convierte la no-detección en límite superior: cuantil de la distribución de máximos de los controles, corregido por el throughput medido, convertido a flujo físico, luminosidad de H α y tasa de acreción. Reporta el límite al 99% (poblado empíricamente) como cifra principal y el equivalente 5σ extrapolado con su advertencia; siempre flujo observado y desenrojecido. Cada límite es auditable desde una sola fila de la tabla.

Productos: tables/halpha_upper_limits.csv, figura de contexto vs. literatura, stage_h03_qc.json.

Bloque F — Cierre

F1 · Productos finales y reporte

```
python scripts/build_report.py
```

Entradas: Todos los QC y productos de las etapas anteriores.

Qué hace: Consolida el run: checklist automática (toda etapa esencial en verde, cadena de hashes entre etapas intacta — detecta mezcla de runs —, convenciones consistentes entre productos), set fijo de siete figuras de publicación, tablas maestras por línea y por método, y un reporte generado de forma determinista: dos ejecuciones producen el mismo resultado, y cada número del reporte es trazable a un QC.

Productos: runs/<RUN_ID>/report/ con run_summary.json, figuras, tablas y report.md.

Parte II · Catálogo de códigos

Función de cada módulo del repositorio: su objetivo, lo que recibe y lo que entrega. Sin detalles de implementación. Los módulos marcados (*) existían antes del plan y fueron reutilizados o extendidos; el resto se creó durante la implementación.

Núcleo común (musepipe/)

`config.py` *

Objetivo: Resolver y validar la configuración del run activo; única fuente de verdad de parámetros.

Entrada: MUSE_RUN_ID o active_run.txt; runs/<RUN_ID>/config/config.json.

Salida: Objeto de configuración validado; error explícito si run y carpeta no coinciden.

`paths.py` *

Objetivo: Rutas estándar de productos por run, para que ningún módulo construya rutas a mano.

Entrada: Identificador del run.

Salida: Rutas de config, stages, tables, plots, logs y report.

`io.py` *

Objetivo: Lectura y escritura uniformes de FITS, CSV y JSON, incluida la malla de longitudes de onda.

Entrada: Rutas de archivos y estructuras de datos.

Salida: Cubos, tablas y QC leídos/escritos con convenciones únicas.

`stats.py` *

Objetivo: Estadística robusta compartida: dispersión por MAD, percentiles finitos, z empírico contra referencias, ruido de continuo.

Entrada: Arreglos numéricos (espectros, mapas, distribuciones de controles).

Salida: Escalares y perfiles estadísticos robustos.

`spectral.py` *

Objetivo: Operaciones espectrales comunes: índices de canal por longitud de onda, máscaras espectrales, continuo por mediana móvil, flujo integrado de línea.

Entrada: Malla de longitudes de onda, espectros 1D, máscaras.

Salida: Índices, máscaras, continuo estimado y flujos integrados.

`apertures.py` *

Objetivo: Aperturas y posiciones de control al mismo radio; suma/promedio de espectros en cajas.

Entrada: Cubo o mapa, posición central, geometría de apertura.

Salida: Espectros por apertura y lista de posiciones de control.

`localfit.py` *

Objetivo: Ajuste y sustracción de superficies locales suaves alrededor de una posición (el fondo del método de control).

Entrada: Cubo, posición del objeto, radios de ajuste y máscara.

Salida: Cubo residual, modelo local y coeficientes del ajuste.

stripes.py *

Objetivo: Toda la maquinaria de geometría y corrección de stripes por correlación cruzada.

Entrada: Cubo apilado, configuración de orientación/rangos de stripes.

Salida: Cubo corregido, mapas de desplazamiento y métricas del patrón.

injection.py

Objetivo: Inyección de fuentes sintéticas (PSF medida + espectro dado) en cubos, independiente del extractor; base de toda la validación de sensibilidad.

Entrada: Cubo base, catálogo de fuentes sintéticas (posición, espectro), psf_model.

Salida: Cubo clonado con las fuentes inyectadas y registro de la verdad inyectada.

psf.py

Objetivo: Ajuste, suavizado y evaluación del modelo de PSF cromática, con su normalización única.

Entrada: Cubo, posición de la primaria, máscaras de fuentes, configuración de bins.

Salida: Función psf_model(λ , dy, dx) y parámetros por bin con incertidumbres.

report.py

Objetivo: Consolidación del run: checklist, figuras estándar y reporte determinista.

Entrada: QC y productos de todas las etapas del run.

Salida: run_summary.json, figuras de publicación, tablas maestras y report.md.

Reducción (musepipe/reduction/)

esorex_driver.py

Objetivo: Orquestrar la reducción ESO desde datos crudos: inventario por headers, construcción y validación de SOF, ejecución recipe a recipe con control de errores.

Entrada: Directorio de raws + calibraciones; configuración del run.

Salida: Cubo reducido con DATA+STAT, pixtables, logs y QC de la reducción.

verify.py

Objetivo: Las seis verificaciones de la reducción cruda (STAT, estándar, WCS, comparación con el ADP, máscara de cielo).

Entrada: Cubo reducido y cubo ADP de referencia.

Salida: Métricas numéricas y figuras; semáforo por verificación en el QC.

sky_zap.py

Objetivo: Decidir por métricas si hacen falta correcciones de cielo y aplicar ZAP con máscara de fuentes segura.

Entrada: Cubo (A1 o ADP), posiciones de las fuentes.

Salida: Decisión documentada; cubo limpio y máscara si se aplicó.

verify_zap.py

Objetivo: Verificar que ZAP limpió sin dañar: reducción de residuales, fuentes intactas, H α intacta, eigenespectras no estelares, STAT sin tocar.

Entrada: Cubos pre y post ZAP, diagnósticos de ZAP.

Salida: Métricas y figuras por verificación; semáforos en el QC.

`telluric.py`

Objetivo: Decidir si la corrección telúrica aporta y aplicarla protegiendo H α y la banda del láser.

Entrada: Cubo, espectro de la primaria, perfiles atmosféricos.

Salida: Decisión documentada; cubo corregido y transmisión 1D si se aplicó.

Control de calidad (musepipe/qc/)

`cube_qc.py`

Objetivo: Medir la calidad de calibración de cualquier cubo MUSE (longitud de onda, LSF, flujo, cielo, varianza) sin modificarlo; puerta de entrada común de la cadena.

Entrada: Cubo(s) con procedencia, catálogo de skylines, fotometría Gaia de la primaria.

Salida: stage00q_qc.json con factores, incertidumbres y semáforos; figuras por medición.

Etapas de preparación (musepipe/stages/)

`stage01_align.py`

Objetivo: Alineación, recorte y apilado con propagación de varianza.

Entrada: Cubo(s) de entrada según entry_point; configuración de crop y centrado.

Salida: Cubo apilado DATA+STAT y QC con desplazamientos y factor de covarianza.

`stage02_xcorr.py`

Objetivo: Ejecutar la corrección de stripes y publicar la métrica de canales sucios.

Entrada: Cubo apilado de B1.

Salida: Cubo corregido, tabla de métrica por canal, QC.

`stage01c_localize.py`

Objetivo: Medir posiciones y astrometría de las fuentes del campo y la deriva cromática del compañero; fijar las coordenadas canónicas del run.

Entrada: Cubo de B2, predicción astrométrica de la literatura.

Salida: QC con posiciones, separación/PA vs. literatura y veredicto cromático.

`stage_e01_psf.py`

Objetivo: Construir y validar el modelo de PSF cromática de la primaria.

Entrada: Cubo de B2, posiciones de B3, LSF de A4.

Salida: psf_model.json evaluable, parámetros por bin, métrica del anillo del compañero.

Extracción (musepipe/extraction/)

`product.py`

Objetivo: Definir el formato único de espectro extraído (SpectrumProduct) con su lector, escritor y validador; el contrato que hace comparables los métodos.

Entrada: Espectros con errores, flags y metadatos de procedencia.

Salida: Archivos FITS de espectro con esquema versionado y validación al leer.

aperture.py

Objetivo: Método de control: envolver la cadena de sustracción local + apertura en el formato estándar, con errores propagados y corrección de apertura.

Entrada: Cubo de B2, coordenadas de B3, curva de crecimiento de C1.

Salida: spec_aperture_object.fits y QC.

optimal.py

Objetivo: Extracción óptima tipo Horne ponderada por PSF y varianza, en dos variantes de fondo.

Entrada: Cubo con fondo restado, psf_model, STAT, coordenadas.

Salida: spec_optimal_*.fits con varianza analítica y QC.

psffit.py

Objetivo: Método primario: ajuste lineal simultáneo estrella + compañero + fondo, canal a canal, con diagnóstico de condicionamiento y crosstalk.

Entrada: Cubo de B2 sin sustraer, psf_model, posiciones, STAT.

Salida: spec_psffit_object.fits, spec_psffit_star.fits, cubo residual y QC.

Comparación, calibración y H-alfa (musepipe/stages/)

stage_x10_compare.py

Objetivo: Árbitro entre métodos: comparación por bandas congeladas con errores de diferencia medidos en controles, y veredicto automático de consistencia.

Entrada: Espectros en formato estándar de todos los métodos y sus controles.

Salida: Tabla de comparación, veredicto en QC y figura panel.

stage_x11_calibrate.py

Objetivo: Producir el espectro final calibrado con presupuesto de errores completo, aplicando solo correcciones medidas aguas arriba (con auditoría anti doble conteo).

Entrada: Espectro canónico, factores de A4, sistemáticos de A2/A3/C1.

Salida: spec_final_object.fits y QC con el presupuesto término a término.

stage_h01_halpha.py

Objetivo: Detección de H α con criterio congelado: matched filter multi-plantilla, significancia empírica y corrección look-elsewhere por método.

Entrada: Espectros calibrados con controles, LSF, velocidad sistémica.

Salida: Tabla de detección por método, veredicto en QC, figuras.

stage_h02_artifacts.py

Objetivo: Batería fija de pruebas de artefactos que toda señal (o no-detección) debe superar.

Entrada: Resultado de E1, cubos residuales, métricas instrumentales de B2/A4.

Salida: Veredicto por prueba y global en QC, figuras.

stage_h03_limits.py

Objetivo: Convertir una no-detección en límites superiores físicos auditables (flujo, luminosidad, tasa de acreción).

Entrada: Distribuciones de controles (E1), throughput (E4), calibración (D2), parámetros físicos con cita.

Salida: Tabla de límites por método y combinado, figura de contexto, QC.

`stage_h04_injection.py`

Objetivo: Medir sensibilidad, throughput y sesgos de toda la cadena por método, mediante inyección-recuperación en clones aislados.

Entrada: Cubo base, psf_model, extractores, grilla de inyección.

Salida: Curvas de throughput/completitud por método, tabla de sesgos, QC.

Scripts de entrada (scripts/)

`reduce_raw.sh`

Objetivo: Punto de entrada único de la reducción cruda (fases 0–3 + verificación).

Entrada: Configuración del run; datos crudos ESO.

Salida: Reducción completa con logs y QC.

`sky_zap.sh` / `telluric.sh`

Objetivo: Fachadas de línea de comandos reproducibles para las etapas condicionales A2 y A3.

Entrada: Cubo de entrada y configuración.

Salida: Decisión y, si aplica, cubo corregido con QC.

`build_report.py`

Objetivo: Regenerar el paquete final del run con un solo comando.

Entrada: QC y productos del run.

Salida: Carpeta report/ completa y determinista.

Fin del documento. Para el detalle normativo de cada etapa (límites, verificaciones numéricas, protocolos de parada), ver las especificaciones docs/spec/_codex_.md y el plan maestro.